Department of Computer Engineering

Technical Interview Question & Model Answer Bank

Academic Year	**2026-2027**	**Class**	**4th year**
Subject/Expert Area	**Object Oriented Programming**		
Expert Name	**Dr. A. S. Jumde**		

Sr. No.	Question	Model Answer	Practical Example
1.	**What is Object-Oriented Programming?**	Object-Oriented Programming (OOP) is a programming paradigm in which programs are designed using objects and classes. An object contains data (attributes) and behavior (methods). The four major principles of OOP are: 1. Encapsulation 2. Abstraction 3. Inheritance 4. Polymorphism	

2.	**What is a class?**	A class is a blueprint or template used to create objects. It defines the properties and behaviors that its objects will have.	class Student {
			String name;
			int age;
			
			void display() {
			System.out.println(name + " " + age);
			}
			}
			
			Here, Student is a class.

3.	**What is an object?**	An object is an instance of a class. It represents a real-world entity and occupies memory.	Student s1 = new Student(); Here, s1 is an object of the Student class.

4.	**What is the difference between a class and an object?**	Class • Blueprint/template • Does not represent a specific entity • Defines properties and methods • Example: Student Object • Instance of a class • Represents a specific entity • Uses those properties and methods • Example: s1	Class • Example: Student Object • Example: s1
5.	**What are the four pillars of OOP?**	: The four major pillars are: 1. Encapsulation – wrapping data and methods together. 2. Abstraction – hiding unnecessary implementation details. 3. Inheritance – acquiring properties and behavior from another class. 4. Polymorphism – allowing one interface/name to have multiple forms.	

6.	**What is encapsulation?**	Encapsulation means bundling data and methods together inside a class and controlling access to the data.
		Usually, data members are declared private and accessed through public methods.
			class Account {
			private double balance;
			
			public void setBalance(double balance) {
			this.balance = balance;
			}
			
			public double getBalance() {
			return balance;
			}
			}

			This prevents direct access to balance.

7.	**What is the advantage of encapsulation?**	Encapsulation provides:
		1. Data protection
		2. Controlled access
		3. Better maintainability
		4. Improved security
		5. Reduced dependency between classes
			For example, we can validate a value before storing it.
			
			public void setAge(int age) {
			if (age >= 0)
			this.age = age;
			}

8.	**What is abstraction?**	Abstraction means hiding implementation details and showing only the essential features to the user.	For example, when we use an ATM, we know how to withdraw money, but we don't need to know the internal implementation. In Java, abstraction can be achieved using: • Abstract classes Interfaces
9.	**What is the difference between abstraction and encapsulation?**	Abstraction 1. Hides implementation details 2. Focuses on what an object does 3. Achieved using abstract classes/interfaces 4. Helps reduce complexity Encapsulation 1. Protects/bundles data 2. Focuses on how data is accessed/protected 3. Commonly achieved using access modifiers 4. Helps achieve data hiding	

10.	**What is inheritance?**	Inheritance is a mechanism through which one class acquires properties and methods of another class.	class Person {
			void display() {
			
			System.out.println("Person");
			}

		In Java, inheritance is implemented using extends.	}
			
			class Employee extends Person {
			}
			
			Here, Employee inherits from Person.

11.	**What are the types of inheritance?**	Common types are: 1. Single inheritance 2. Multilevel inheritance 3. Hierarchical inheritance 4. Multiple inheritance 5. Hybrid inheritance Java supports single, multilevel, and hierarchical inheritance through classes. Java does not support multiple inheritance through classes because it can create ambiguity. Multiple inheritance of type can be achieved using interfaces.	

12.	**What is single inheritance?**	When one child class inherits from one parent class, it is called single inheritance.	class Person {
			void showPerson() {
			
			System.out.println("Person");
			}
			}
			
			class Employee extends Person {
			void showEmployee() {
			
			System.out.println("Employee");
			}
			}
			
			Relationship:
			
			Person
			↓
			Employee

13.	**What is multilevel inheritance?**	When inheritance occurs in multiple levels, it is called multilevel inheritance.	class Animal {
			}
			
			class Dog extends Animal {
			}
			
			class Puppy extends Dog {
			}
			
			Relationship:
			
			Animal
			↓
			Dog
			↓
			Puppy

14.	**What is polymorphism?**	: Polymorphism means "many forms." It allows the same method name or interface to behave differently depending on the situation. There are two common forms: 1. Compile-time polymorphism – method overloading 2. Runtime polymorphism – method overriding	

15.	**What is method overloading?**	Method overloading occurs when multiple methods have the same name but different parameter lists in the same class.	class Calculator {
			
			int add(int a, int b) {
			return a + b;
			}
			int add(int a, int b, int c) {
			return a + b + c;
			}
			}
			
			The methods have the same

			name but different numbers of parameters.

16.	**What is method overriding?**	Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.	class Animal {
			void sound() {
			System.out.println("Animal makes sound");
			}
			}
			class Dog extends Animal {
			@Override
			void sound() {
			System.out.println("Dog barks");
			}
			}
			
			Here, Dog overrides sound().

17.	**What is the difference between overloading and overriding?**	Overloading 1. Same method name 2. Parameters must differ 3. Usually occurs within the same class 4. Compile-time polymorphism 5. Return type alone cannot differentiate methods Overriding 1. Same method signature 2. Parameters must be the same 3. Occurs between parent and child classes 4. Runtime polymorphism 5. Return type must be compatible	
18.	**What is runtime polymorphism?**	Runtime polymorphism occurs when the method to be executed is determined at runtime. It is achieved through method overriding and a parent-class	Animal a = new Dog(); a.sound(); If Dog overrides sound(), the Dog version executes.

		reference referring to a child-class object.	

19.	**What is compile-time polymorphism?**	Answer: Compile-time polymorphism occurs when the method call is resolved by the compiler.
		
		The most common example is method overloading.
			class Calculator {
			
			int add(int a, int b) {
			return a + b;
			}
			
			double add(double a, double b) {
			return a + b;
			}
			}
			
			The compiler determines which add() method should be called.

20.	**What is a constructor?**	A constructor is a special member of a class that is used to initialize objects.
		
		In Java:
		1. Constructor has the same name as the class.
		2. It does not have a return type.
		3. It is automatically called when an object is created.
			class Student {
			Student() {
			
			System.out.println("Constructor called");
			}
			}
			
			Student s = new Student();

21.	**What is constructor overloading?**	Constructor overloading means having multiple constructors with different parameter lists.	class Student {
			
			Student() {
			
			System.out.println("Default constructor");
			}
			
			Student(String name) {
			System.out.println("Name: " + name);
			}
			}

			Both constructors have the same name but different parameters.

22.	**What is constructor chaining?**	Constructor chaining means calling one constructor from another constructor.
		
		In Java:
		1. this() calls another constructor in the same class.
		2. super() calls a constructor of the parent class.
			Example:
			
			class Person {
			Person() {
			System.out.println("Person constructor");
			}
			}
			
			class Employee extends Person {
			Employee() {
			super();
			
			System.out.println("Employee constructor");
			}
			}
			
			When an Employee object is created, the parent constructor executes before the child constructor.

23.	**What is the super keyword?**	super refers to the immediate parent class.
		
		It can be used to:
		1. Access parent class variables
		2. Call parent class methods
		3. Call parent class constructors
			Example:
			
			class Person {
			String name = "Person";
			}
			
			class Employee extends Person {
			String name = "Employee";
			
			void display() {
			
			System.out.println(super.name);
			}
			}
			
			Output:
			
			Person

24.	**What is the this keyword?**	this refers to the current object.	class Student { String name;

		It is commonly used to distinguish instance variables from local variables.	Student(String name) {
			this.name = name;
			}
			}
			
			Here, this.name refers to the instance variable.

25.	**What is an abstract class?**	An abstract class is a class declared using the abstract keyword. It can contain both abstract and concrete methods.	abstract class Animal {
			
			abstract void sound();
			
			void eat() {
			System.out.println("Animal eats");
			}
			}
			
			An abstract class cannot be directly instantiated.
			
			// Animal a = new Animal(); // Not allowed

26.	**What is an interface?**	An interface defines a contract that implementing classes must follow.	interface Vehicle {
			void start();
			}
			
			class Car implements Vehicle {
			public void start() {
			System.out.println("Car starts");
			}
			}
			
			A class implements an interface using the implements keyword.

27.	**What is the difference between an abstract class and an interface?**	**Abstract Class** Declared using abstract class Can have instance variables Can have constructors Can contain abstract and concrete methods	

		A class extends one class **Interface** Declared using interface Fields are constants by default Cannot have constructors Can define abstract methods and, in modern Java, also default/static/private methods A class can implement multiple interfaces	

28.	**Does Java support multiple inheritance?**	Java does not support multiple inheritance using classes.	For example, this is not allowed:
			
			class C extends A, B {
			}
			
			One reason is the diamond problem, where Java could face ambiguity if both parent classes provide the same method.
			
			However, Java allows a class to implement multiple interfaces:
			
			class C implements A, B {
			}

29.	**What is association, aggregation, and composition?**	All three represent relationships between objects. Association: General relationship between two objects. Teacher ↔ Student Aggregation: A weak has-a relationship. The contained object can exist independently. Department → Teacher A teacher can exist even if the department is removed.	

		Composition: A strong has-a relationship. The contained object is strongly dependent on the containing object. House → Room Conceptually, rooms are part of the house.	

30.	**What is the difference between IS-A and HAS-A relationships?**	IS-A → Inheritance
		class Dog extends Animal {
		}
		
		A Dog IS-A Animal.
		
		HAS-A → Composition/Aggregation
		class Car {
		  Engine engine;
		}
		
		A Car HAS-A Engine.	